
IIC2523

Sistemas Distribuidos

— Hernán F. Valdivieso López —
(2026 - 2 / Clase 09)

Elección de Líder

Temas de la clase

1. Algoritmos de elección de líder.
2. Algoritmo *Bully*.
3. Algoritmo *Raft*: consenso de información a través de un líder.

Algoritmos de elección de líder

Elección de líder

- ◆ Consiste en elegir un nodo/proceso único para desempeñar un **rol particular** en un sistema distribuido:
 - ◆ Coordinador.
 - ◆ Iniciador de algún protocolo/algoritmo.
 - ◆ Ser el servidor para sincronizar relojes.
 - ◆ Acceder a un recurso crítico.
 - ◆ Entre otros.

Elección de líder - Características

- ◆ Cada proceso/nodo debe tener un identificador único que puede ser directamente su PID, algún valor dinámico, alguna propiedad, una combinación de valores, etc.
- ◆ Se espera que todo algoritmo de elección de líder cumple con 2 propiedades:
 - ◆ **Safety**: Nunca hay dos líderes válidos a la vez.
 - ◆ **Liveness**: Eventualmente habrá un líder elegido y si este cae, eventualmente se elige otro. Nunca quedamos estancados sin un líder.

Algoritmos *Bully*

Elección de líder - Algoritmo *Bully*



Elección de líder - Algoritmo *Bully*

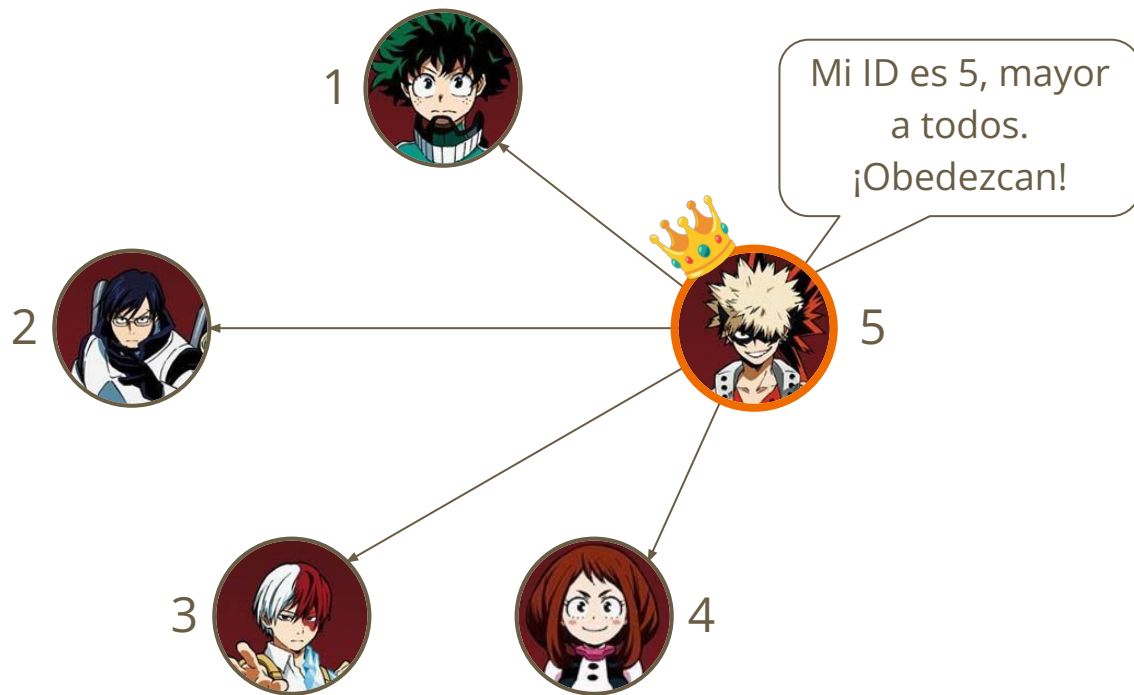
- ◆ Propuesto por Garcia-Molina en 1982.
- ◆ Es un algoritmo **síncrono** que utiliza tiempos de espera (*timeouts*) para detectar fallas en los nodos.
 - ◆ Se establece *delay* máximo de transmisión de mensajes (T_{trans})
 - ◆ Se establece *delay* máximo de procesamiento de mensajes ($T_{process}$)
 - ◆ Si un nodo demora más de $2 \times T_{trans} + T_{process}$ en responder, se asume una falla.
- ◆ Supuestos:
 - ◆ El identificador de cada nodo es un valor fijo que nunca cambia.
 - ◆ Cada nodo sabe qué nodo tienen identificadores más altos y puede comunicarse con todos ellos.

Elección de líder - Algoritmo *Bully* - Funcionamiento

- ◆ Este algoritmo utiliza tres tipos de mensajes:
 - ◆ **Election**: Anunciar una elección y nominarse como candidato.
 - ◆ **Ok**: Respuesta a un mensaje Election para detener la candidatura.
 - ◆ **Coordinator**: Enviado para anunciar la identidad del nodo/proceso elegido.

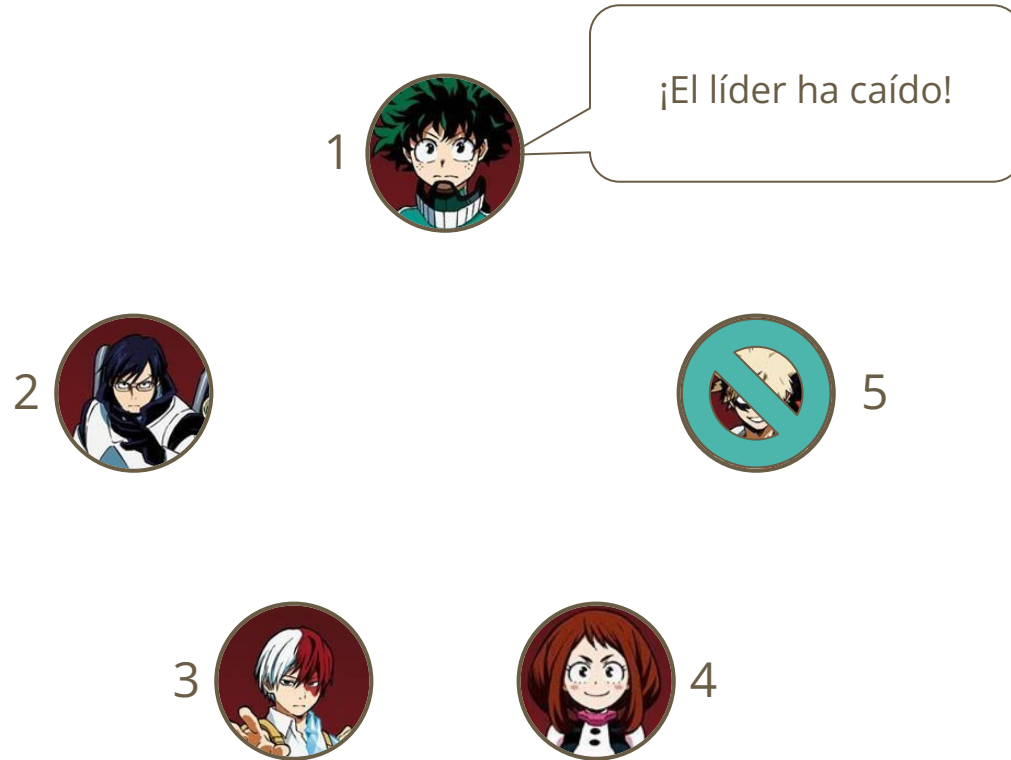
Elección de líder - Algoritmo *Bully* - Funcionamiento

- ◆ **Caso inicial:** como todos saben el identificador de cada nodo. Aquel con mayor identificador manda mensaje Coordinator para avisar que será el líder



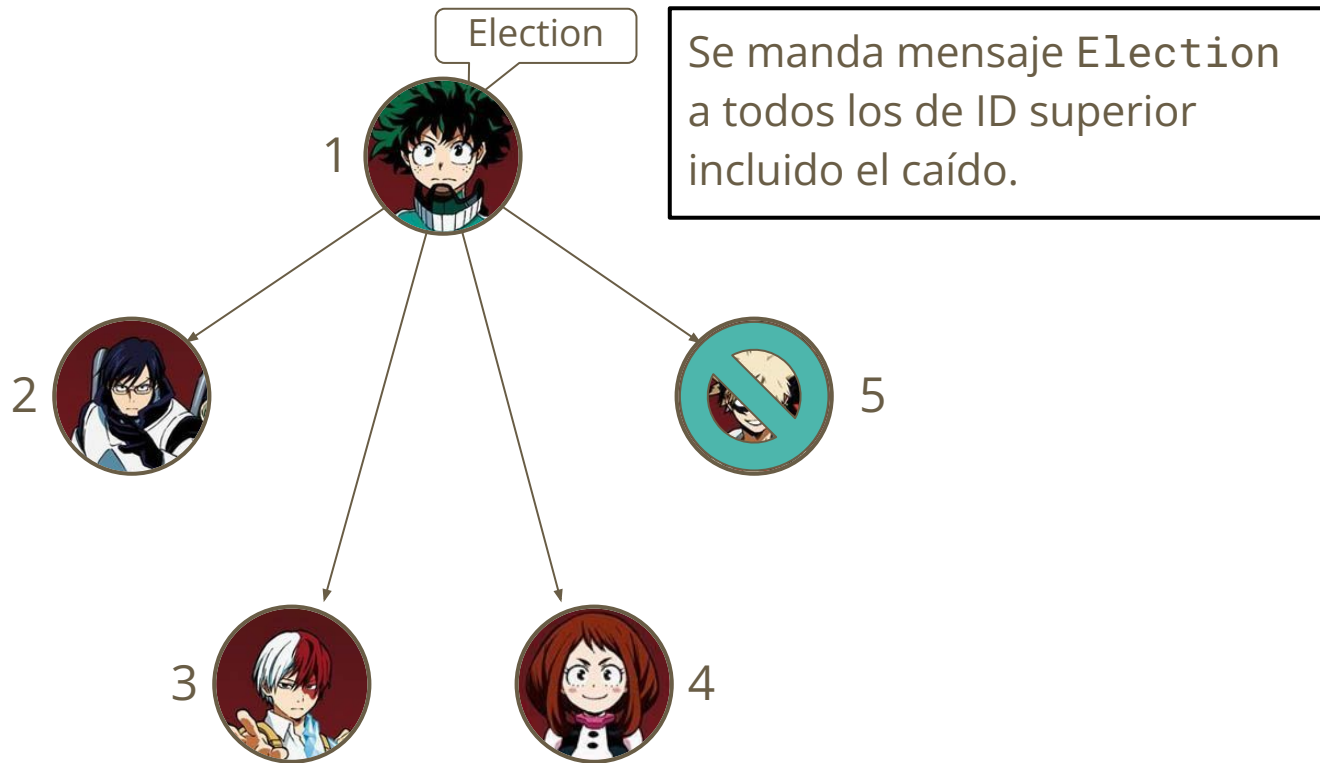
Elección de líder - Algoritmo *Bully* - Funcionamiento

- ◆ **Caso el líder falla/muere.** Algún nodo se da cuenta de esto y comienza elección.



Elección de líder - Algoritmo *Bully* - Funcionamiento

- ◆ **Caso el líder falla/muere.** Algún nodo se da cuenta de esto y comienza elección.



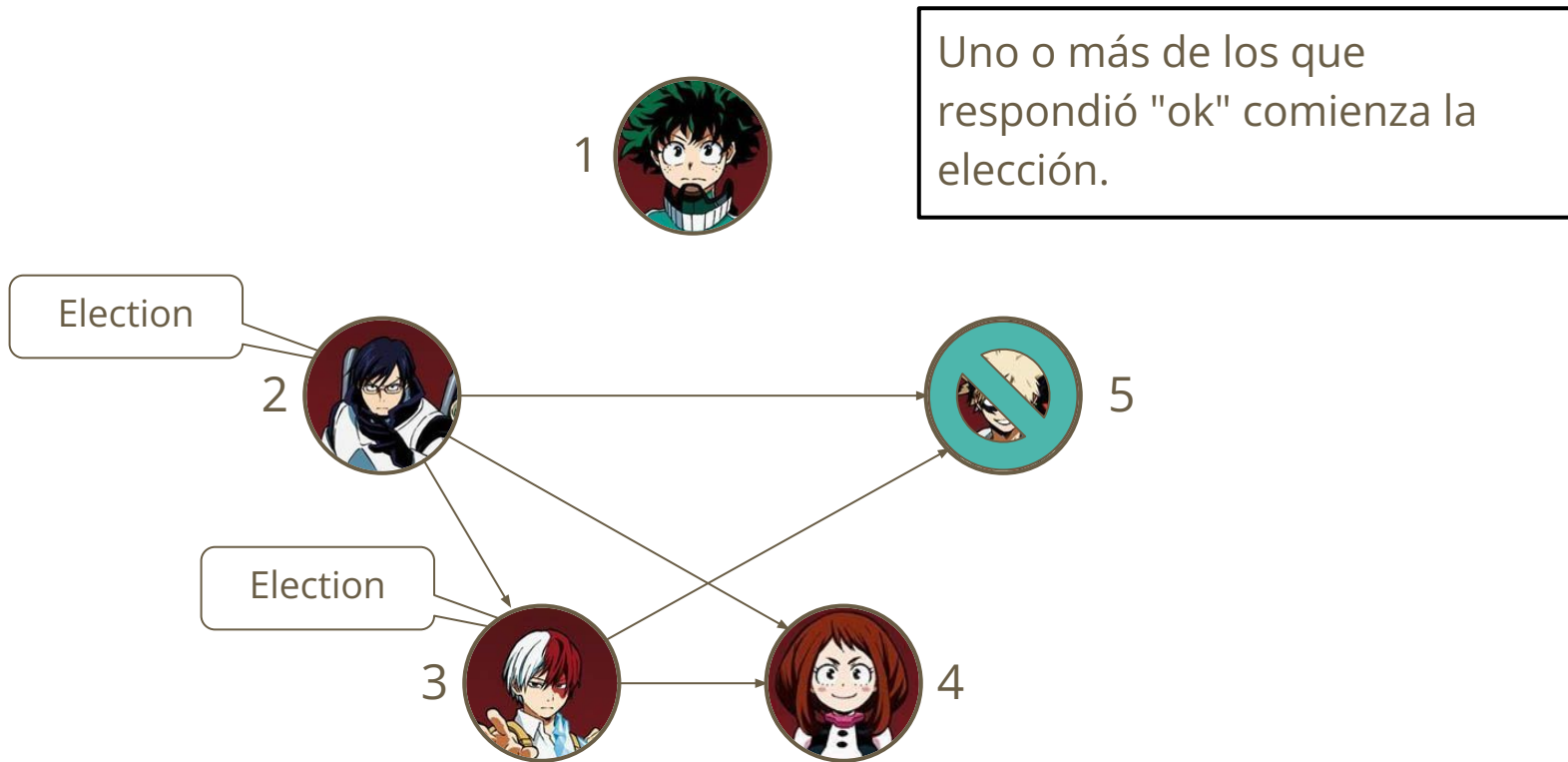
Elección de líder - Algoritmo *Bully* - Funcionamiento

- ◆ **Caso el líder falla/muere.** Algún nodo se da cuenta de esto y comienza elección.



Elección de líder - Algoritmo *Bully* - Funcionamiento

◆ **Caso el líder falla/muere.** Algún nodo se da cuenta de esto y comienza elección.



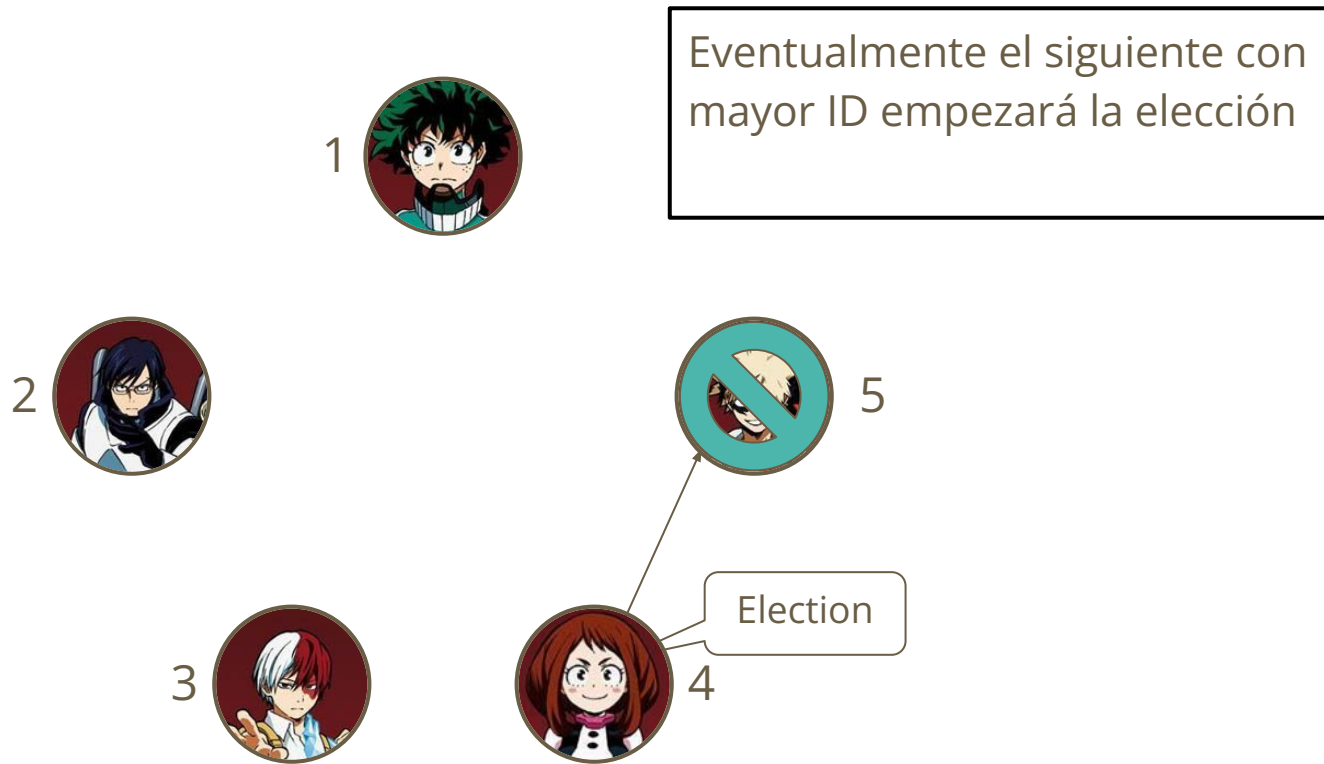
Elección de líder - Algoritmo *Bully* - Funcionamiento

- ◆ **Caso el líder falla/muere.** Algún nodo se da cuenta de esto y comienza elección.



Elección de líder - Algoritmo *Bully* - Funcionamiento

- ◆ **Caso el líder falla/muere.** Algún nodo se da cuenta de esto y comienza elección.



Elección de líder - Algoritmo *Bully* - Funcionamiento

- ◆ **Caso el líder falla/muere.** Algún nodo se da cuenta de esto y comienza elección.



Elección de líder - Algoritmo *Bully* - Resumen

- ◆ Nodo con mayor identificador es líder siempre.
- ◆ Si líder se cae, algún nodo comienza la elección. Ese nodo contacta **exclusivamente** a todos con ID superior.
 - ◆ Si al menos uno con ID superior responde (Ok), entonces el candidato se retira de la elección. El nodo que respondió comienza la elección eventualmente.
 - ◆ Si ningún nodo con ID superior responde, entonces el que comenzó la elección está capacitado de ser líder y avisa a todos los nodos activos.

Elección de líder - Algoritmo *Bully* - Observaciones

- ◆ Es un algoritmo que, en el peor caso, se mandan demasiados mensajes.
 - ◆ El nodo con menor id manda $N-1$ mensajes de Election.
 - ◆ El siguiente manda $N-2$ mensajes de Election.
 - ◆ El siguiente manda $N-3$ mensajes de Election.
 - ◆ Hasta llegar al segundo mejor nodo que manda 1 mensaje de Election.

Elección de líder - Algoritmo *Bully* - Observaciones

- ◆ Es un algoritmo que, en el peor caso, se mandan demasiados mensajes.
 - ◆ El nodo con menor id manda $N-1$ mensajes de Election.
 - ◆ El siguiente manda $N-2$ mensajes de Election.
 - ◆ El siguiente manda $N-3$ mensajes de Election.
 - ◆ Hasta llegar al segundo mejor nodo que manda 1 mensaje de Election.
- ◆ En una red distribuida, no es garantía que siempre se sepa todos los nodos participantes y sus ID, por lo cual puede ser necesario enviar Election a muchos más nodos (incluso con ID menor) para tener mayor seguridad de ser el con mayor ID.
 - ◆ Si un nodo con ID mayor envía Election a un nodo con ID menor, simplemente el con ID menor no responde.

Raft

**Consenso de datos a través de
la elección de un líder
democráticamente**

Raft - Introducción

- ◆ Propuesto el 2014 por Diego Ongaro y John Ousterhout.
- ◆ Trabajo original: [In Search of an Understandable Consensus Algorithm](#)
- ◆ Citando su paper:

Raft is a consensus algorithm for managing a replicated log. It produces a result equivalent to (multi-)Paxos, and it is as efficient as Paxos, but its structure is different from Paxos; this makes Raft more understandable than Paxos...

Raft - Estados y fases

◆ Se proponen 3 estados:

- ◆ **Seguidor (*Follower*)**: estado por defecto, solo responde a peticiones.
- ◆ **Candidato (*candidate*)**: busca votos para ser el líder.
- ◆ **Líder (*leader*)**: encargado de replicar las acciones para lograr consolidar.

◆ Hay 2 fases:

- ◆ Elección de líder.
- ◆ Replicación de *logs*.

Raft - Estados y fases

◆ Se proponen 3 estados:

- ◆ **Seguidor (*Follower*)**: estado por defecto, solo responde a peticiones.
- ◆ **Candidato (*candidate*)**: busca votos para ser el líder.
- ◆ **Líder (*leader*)**: encargado de replicar las acciones para lograr consolidar.

◆ Hay 2 fases:

- ◆ Elección de líder.
- ◆ Replicación de *logs*.

◆ Hay un término que usaremos mucho: "***term***" que lo veremos como el "periodo de mandato" de un líder. Es un número que solo va creciendo.

◆ Todo nodo tiene un registro del *term* que lo compartirá con los demás nodos. El *term* más grande manda en todo momento.

Raft - Fase 1 - Elección de líder

- ◆ Cada nodo tiene un *election timeout* que es un número aleatorio entre 150 a 300 milisegundos. Es un intento de asegurar un número distinto por nodo.
- ◆ **Postulación:** cuando se acaba el *election timeout*, el nodo se postula como candidato.
 - a. Aumenta el "*term*/mandato".
 - b. El nodo vota por sí mismo.
 - c. Le pide a todos los demás nodos que voten por él.
 - d. Si logra mayoría, se vuelve líder.
 - e. Si no logra mayoría, espera reiniciar su *election timeout* y vuelve a paso A.

Raft - Fase 1 - Elección de líder

◆ **Votación:** todo nodo que reciba solicitud de voto, debe responder:

1. Si ya votó en el *term* actual o el *term* del candidato es menor al *term* del votante, **vota rechazo**.
2. Si el nodo votante no tiene *logs* todavía, **vota a favor**.
3. En otro caso, compara los *logs* (registro de acciones) del nodo votante y del candidato.
 - a. Si el último *log* del candidato fue en un *term* mayor al nodo votante, **vota a favor del candidato**.
 - b. Si el último *log* del candidato fue en un *term* igual al nodo votante, **solo se acepta si su *log* es igual o más largo que el nodo votante**.
 - c. Si el último *log* del candidato fue en un *term* menos al nodo votante, **vota rechazo**.

Raft - Fase 1 - Elección de líder

◆ **Votación:** todo nodo que reciba solicitud de voto, debe responder:

1. Si ya votó en el *term* actual o el *term* del candidato es menor al *term* del votante, **vota rechazo**.
2. Si el nodo votante no tiene *logs* todavía, **vota a favor**.
3. En otro caso, compara los *logs* (registro de acciones) del nodo votante y del candidato.
 - a. Si el último *log* del candidato fue en un *term* mayor al nodo votante, **vota a favor del candidato**.
 - b. Si el último *log* del candidato fue en un *term* igual al nodo votante, **solo se acepta si su *log* es igual o más largo que el nodo votante**.
 - c. Si el último *log* del candidato fue en un *term* menos al nodo votante, **vota rechazo**.

En las casos que vota a favor, el nodo reinicia su *election timeout*, para no postularse todavía.

Raft - Fase 1 - Elección de líder

Considerando la tupla (A_x, N) la acción X que se registró en el *log* durante el *term* N .

Caso 1 - [estado inicial]



$[(A_1, 1), (A_2, 1), (A_3, 2)]$

Term: 2

Timeout: 150

Nodo B



$[(A_1, 1), (A_2, 1)]$

Term: 2

Timeout: 200



$[]$

Term: 0

Timeout: 210



$[(A_1, 1), (A_2, 1)]$

Term: 1

Timeout: 250

Raft - Fase 1 - Elección de líder

Considerando la tupla (A_x, N) la acción X que se registró en el *log* durante el *term* N .

Caso 1 - El más actualizado comienza votación.



$[(A_1, 1), (A_2, 1), (A_3, 2)]$

Term: 3

Timeout: 150

Nodo B



$[(A_1, 1), (A_2, 1)]$

Term: ~~2~~ 3

Timeout: 200

Nodo D



$[]$

Term: ~~0~~ 3

Timeout: 210

Nodo S



$[(A_1, 1), (A_2, 1)]$

Term: ~~1~~ 3

Timeout: 250

Raft - Fase 1 - Elección de líder

Considerando la tupla (A_x, N) la acción X que se registró en el *log* durante el *term* N .

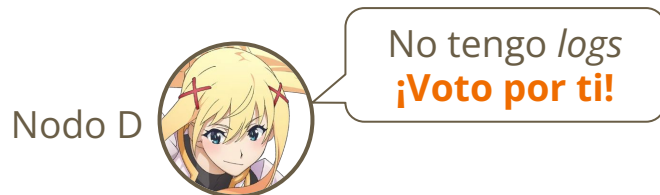
Caso 1 - El más actualizado comienza votación.



$[(A_1, 1), (A_2, 1), (A_3, 2)]$

Term: 3

Timeout: 150



$[\]$

Term: ~~0~~ 3

Timeout: 210

Nodo B



Tu último registro está
en *term* 2 vs el mio que
es *term* 1.

¡Voto por ti!

$[(A_1, 1), (A_2, 1)]$

Term: ~~2~~ 3

Timeout: 200



¡Voto por ti!

$[(A_1, 1), (A_2, 1)]$

Term: ~~1~~ 3

Timeout: 250

Raft - Fase 1 - Elección de líder

Considerando la tupla (A_x, N) la acción X que se registró en el *log* durante el *term* N .

Caso 2 - El con *term* fuera de lugar comienza votación.

Nodo C



$[(A_1, 1), (A_2, 1), (A_3, 2)]$

Term: 2

Timeout: 200

Nodo B



$[(A_1, 1), (A_2, 1)]$

Term: 2

Timeout: 210

Nodo D



$[\]$

Term: 0

Timeout: 150

Nodo S



$[(A_1, 1), (A_2, 1)]$

Term: 2

Timeout: 250

Raft - Fase 1 - Elección de líder

Considerando la tupla (A_x, N) la acción X que se registró en el *log* durante el *term* N .

Caso 2 - El con *term* fuera de lugar comienza votación.



$[(A_1, 1), (A_2, 1), (A_3, 2)]$

Term: 2

Timeout: 200

Nodo B



$[(A_1, 1), (A_2, 1)]$

Term: 2

Timeout: 210

Nodo D



Voten por mi

$[\]$

Term: 1

Timeout: 150

Nodo S



$[(A_1, 1), (A_2, 1)]$

Term: 2

Timeout: 250

Raft - Fase 1 - Elección de líder

Considerando la tupla (A_x, N) la acción X que se registró en el *log* durante el *term* N .

Caso 2 - El con *term* fuera de lugar comienza votación.

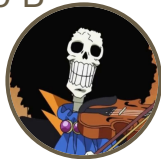
Nodo C



Tu *term* actual es menor al mío.
¡Rechazado!

$[(A_1, 1), (A_2, 1), (A_3, 2)]$
Term: 2
Timeout: 200

Nodo B



Tu *term* actual es menor al mío.
¡Rechazado!

$[(A_1, 1), (A_2, 1)]$
Term: 2
Timeout: 210

Nodo D



Voten por mi

$[\]$
Term: ~~1~~ 2
Timeout: 150

Nodo S



Tu *term* actual es menor al mío.
¡Rechazado!

$[(A_1, 1), (A_2, 1)]$
Term: 2
Timeout: 250

Raft - Fase 1 - Elección de líder

Considerando la tupla (A_x, N) la acción X que se registró en el *log* durante el *term* N .

Caso 3 - *Log* no tan actualizado



$[(A_1, 1), (A_2, 1), (A_3, 2)]$

Term: 2

Timeout: 220

Nodo B



$[(A_1, 1), (A_2, 1)]$

Term: 2

Timeout: 150



$[]$

Term: 0

Timeout: 210



$[(A_1, 1), (A_2, 1)]$

Term: 2

Timeout: 250

Raft - Fase 1 - Elección de líder

Considerando la tupla (A_x, N) la acción X que se registró en el *log* durante el *term* N .

Caso 3 - *Log* no tan actualizado

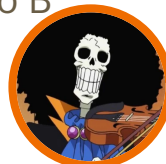


$[(A_1, 1), (A_2, 1), (A_3, 2)]$

~~Term: 3~~

Timeout: 220

Nodo B



Voten por mi

$[(A_1, 1), (A_2, 1)]$

~~Term: 3~~

Timeout: 150

Nodo D



$[]$

~~Term: 3~~

Timeout: 210

Nodo S



$[(A_1, 1), (A_2, 1)]$

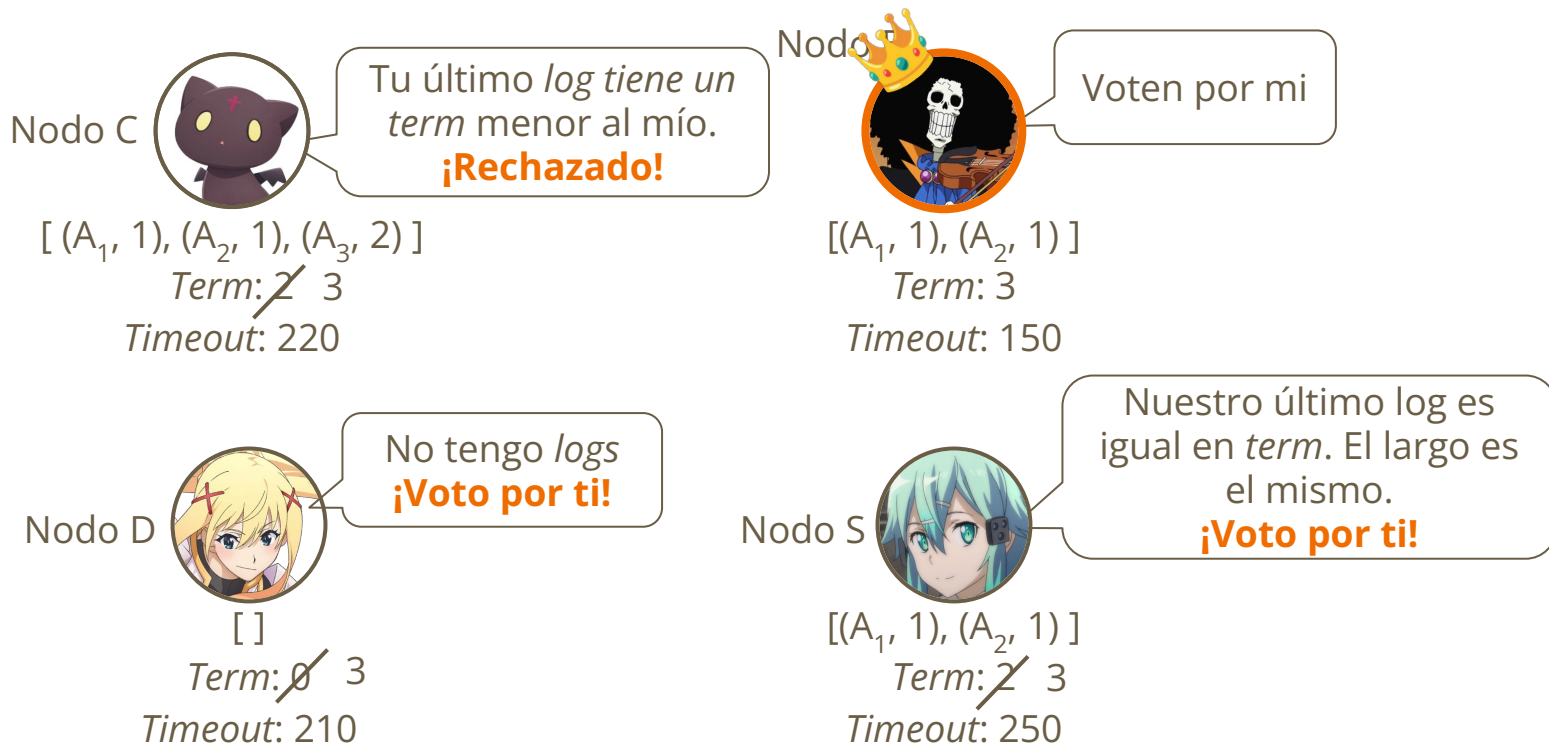
~~Term: 3~~

Timeout: 250

Raft - Fase 1 - Elección de líder

Considerando la tupla (A_x, N) la acción X que se registró en el *log* durante el *term* N .

Caso 3 - *Log* no tan actualizado



Raft - Fase 1 - Elección de líder

Considerando la tupla (A_x, N) la acción X que se registró en el *log* durante el *term* N .

Caso 4 - El con *log* vacío comienza votación.



$[(A_1, 1), (A_2, 1), (A_3, 2)]$

Term: 2

Timeout: 200

Nodo B



$[(A_1, 1), (A_2, 1)]$

Term: 2

Timeout: 210



$[]$

Term: 2

Timeout: 150



$[(A_1, 1), (A_2, 1)]$

Term: 2

Timeout: 250

Raft - Fase 1 - Elección de líder

Considerando la tupla (A_x, N) la acción X que se registró en el *log* durante el *term* N .

Caso 4 - El con *log* vacío comienza votación.



$[(A_1, 1), (A_2, 1), (A_3, 2)]$

~~Term: 3~~

Timeout: 200

Nodo B



$[(A_1, 1), (A_2, 1)]$

~~Term: 3~~

Timeout: 210

Nodo D



Voten por mi

$[]$

Term: 3

Timeout: 150

Nodo S



$[(A_1, 1), (A_2, 1)]$

~~Term: 3~~

Timeout: 250

Raft - Fase 1 - Elección de líder

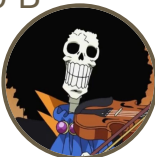
Considerando la tupla (A_x, N) la acción X que se registró en el *log* durante el *term* N .

Caso 4 - El con *log* vacío comienza votación.

Nodo C 

Tengo datos cuyo *term* es más actual que el tuyo
¡Rechazado!

$[(A_1, 1), (A_2, 1), (A_3, 2)]$
~~Term: 3~~
Timeout: 200

Nodo B 

Tengo datos cuyo *term* es más actual que el tuyo
¡Rechazado!

$[(A_1, 1), (A_2, 1)]$
~~Term: 3~~
Timeout: 210

Nodo D 

Voten por mi

$[\]$
Term: 3
Timeout: 150

Nodo S 

Tengo datos cuyo *term* es más actual que el tuyo
¡Rechazado!

$[(A_1, 1), (A_2, 1)]$
~~Term: 3~~
Timeout: 250

Raft - Fase 1 - Elección de líder (Resumen)

- ◆ Todos parten como *seguidores* y votan una vez por *term*/mandato.
- ◆ **Postulación:** se acaba el *election timeout* de un nodo y se convierte en candidato.
- ◆ **Votación:** solo se rechaza si el nodo candidato está menos actualizado que el nodo votante. En caso de empate, se acepta al nodo candidato.
- ◆ **Elección de líder:** el candidato logra tener mayoría de votos.



Raft - Fase 1 - Elección de líder (Peeero)

Siendo el líder...

🤔 ¿Cómo evito perder el trono, es decir, cómo evito que empiece otra votación? 🤔

Heartbeat: el líder manda constantemente un mensaje a todos los seguidores para reiniciar su *election timeout*.

Raft - Fase 1 - Elección de líder (Peeero)

Siendo el líder...

🤔 ¿Cómo evito perder el trono, es decir, cómo evito que empiece otra votación? 🤔

Heartbeat: el líder manda constantemente un mensaje a todos los seguidores para reiniciar su *election timeout*.

🤔 Y si me desconecto y vuelvo... ¿cómo me doy cuenta que empezó una votación? 🤔

Cuando se comienza una votación, el *term* aumenta. Cualquier nodo que ve un *term* mayor al suyo se vuelve seguidor de inmediato y actualiza su *term* al valor más grande recibido.

Raft - Fase 2 - Replicación de *logs*

Logs

- ◆ Lista de tuplas. Cada tupla tiene la operación y el *term* en que se creó la operación.

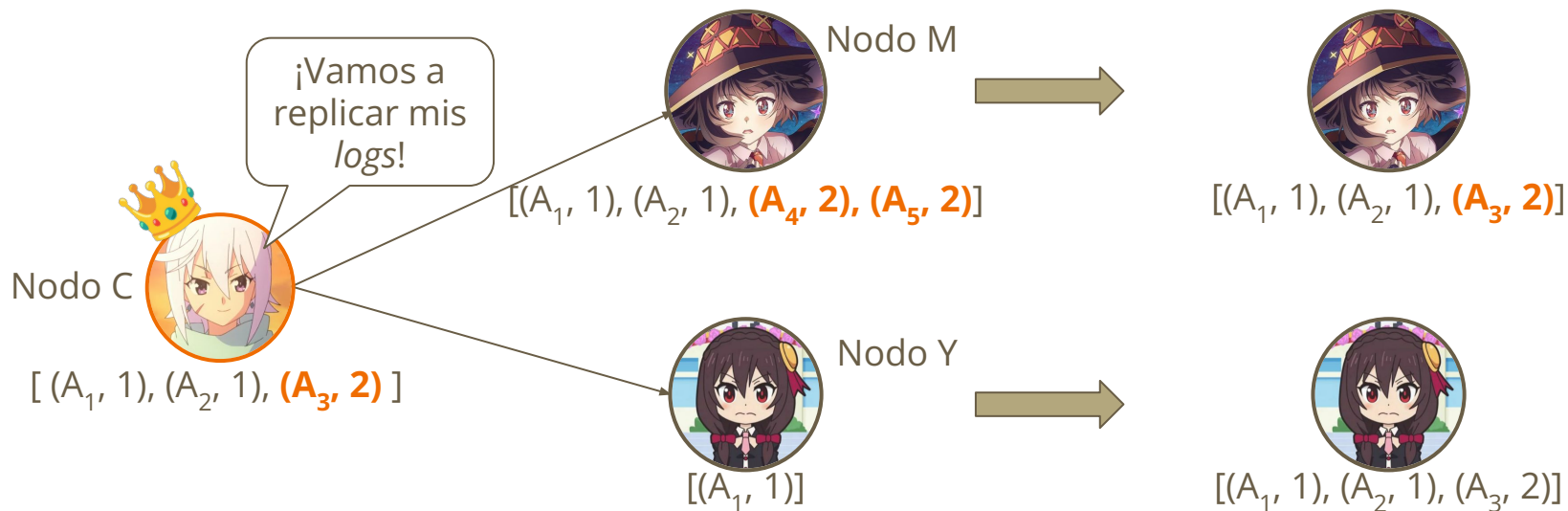
Replicación

- ◆ El líder recibe operaciones, la guarda en su *log*.
- ◆ Si un seguidor recibe una operación, la redirige al líder.
- ◆ Líder intenta replicar su *log* en seguidores.
- ◆ Si una posición del *log* tiene conflicto entre la tupla del líder y del seguidor, gana la tupla del líder.

Raft - Fase 2 - Replicación de *logs*

Replicar

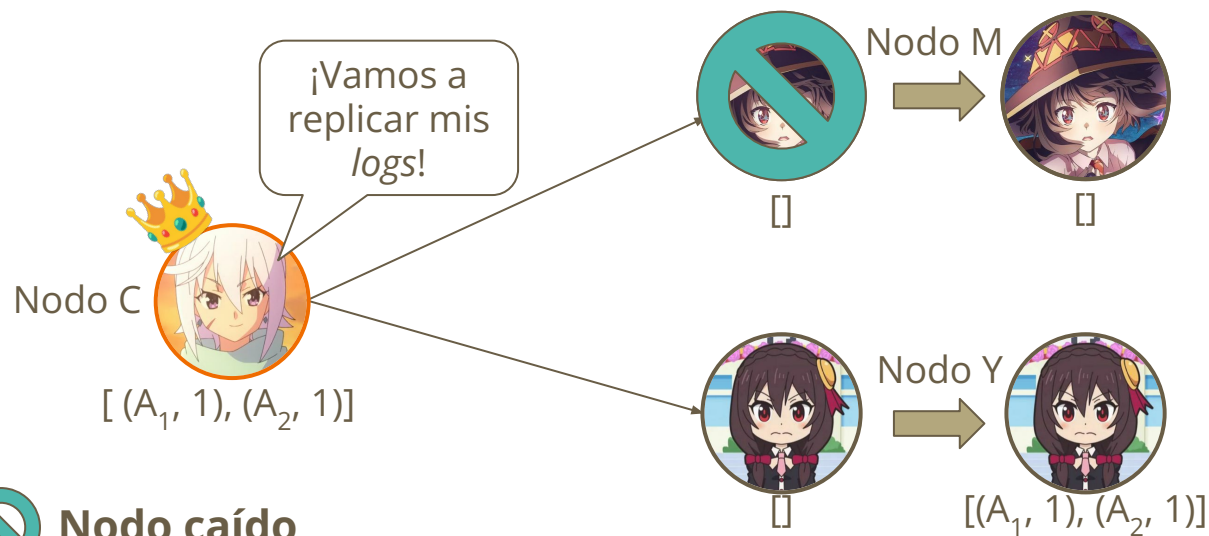
- ◆ Líder intenta replicar su *log* en seguidores.
- ◆ Si una posición del *log* tiene conflicto entre la tupla del líder y del seguidor, gana la tupla del líder.



Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**



Las acciones A_1 y A_2 serán consolidadas.

Las tiene el líder y un seguidor (2 de nodos de 3),



 **Nodo caído**

Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- ◆ Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**

Term 1

A-1		
B-5		
C-4		
D-3		
E-2		

Cada nodo tiene un ID (la letra) y un *election timeout* distinto (el número después de la letra)

Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- ◆ Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**

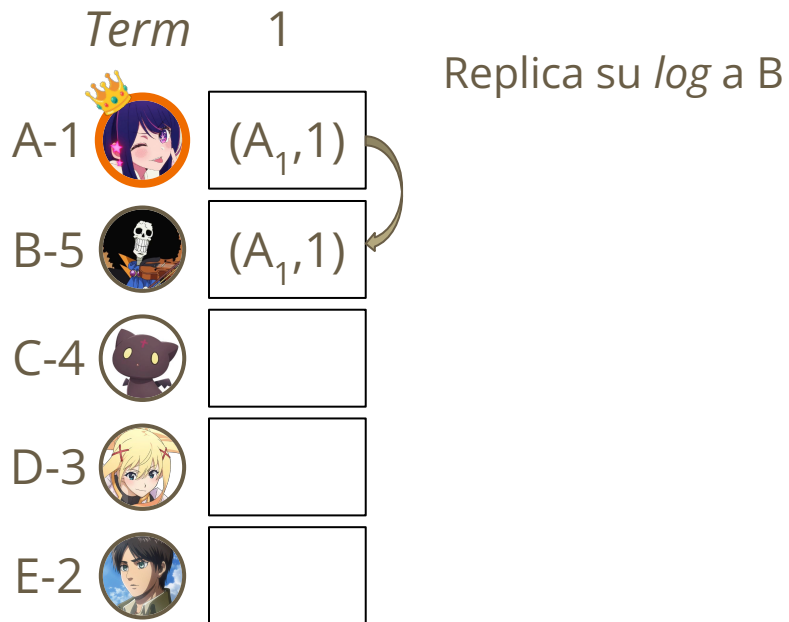
Term 1	
A-1 	(A ₁ ,1)
B-5 	
C-4 	
D-3 	
E-2 	

A es líder por menor *timeout* y le llega una acción.

Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- ◆ Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**



Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**



Term	1	2
A-1 	(A ₁ ,1)	(A ₁ ,1)
B-5 	(A ₁ ,1)	(A ₁ ,1)
C-4 		
D-3 		
E-2 		

A se cae temporalmente.

E, segundo con menor *timeout*, logra mayoría gracias al voto de C y E.

Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**



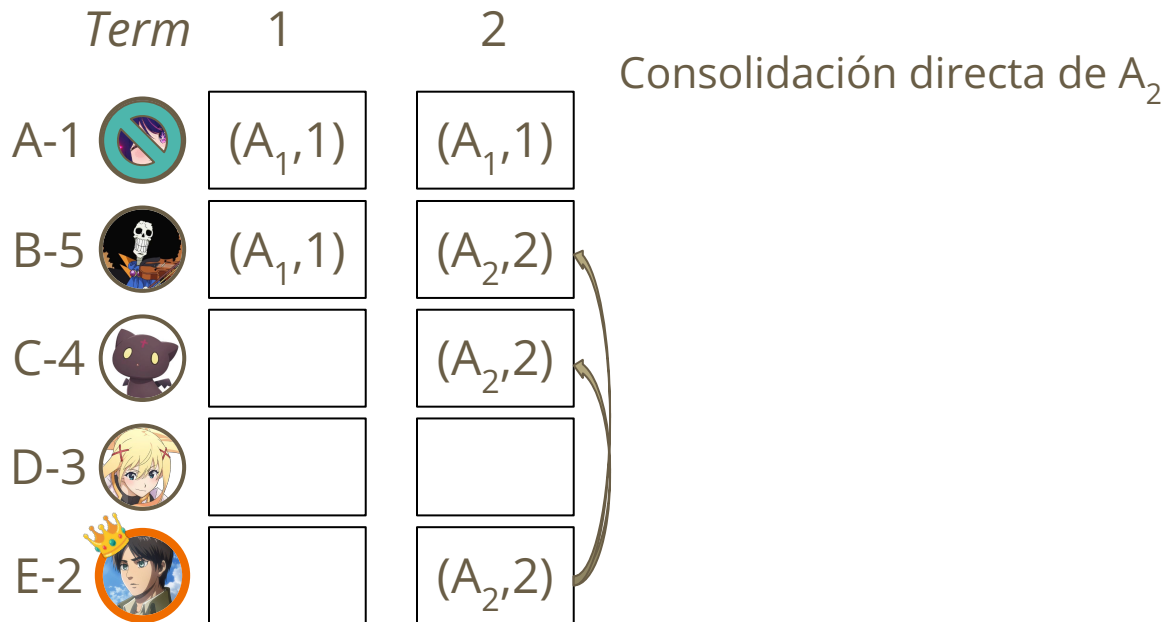
Term	1	2
A-1 	(A ₁ ,1)	(A ₁ ,1)
B-5 	(A ₁ ,1)	(A ₁ ,1)
C-4 		
D-3 		
E-2 		(A ₂ ,2)

Llega acción a E.

Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**



Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**



Term	1	2
A-1 	(A ₁ ,1)	(A ₁ ,1)
B-5 	(A ₁ ,1)	(A ₂ ,2)
C-4 		(A ₂ ,2)
D-3 		
E-2 		(A ₂ ,2)

Se cae E y A vuelve.

El siguiente líder necesita 3 votos.

A y D no pueden ser elegidos como líderes porque B y C no aceptarán su candidatura,

→ **A₂ es una operación consolidada.**

Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**



Term	1	2
A-1 	(A ₁ ,1)	(A ₁ ,1)
B-5 	(A ₁ ,1)	(A ₁ ,1)
C-4 		
D-3 		
E-2 		(A ₂ ,2)

Volvamos a este caso.

A no alcanzó a consolidar la primera acción.

E, siendo líder, le llega la acción A₂

Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**



	Term	1	2
A-1		(A ₁ ,1)	(A ₁ ,1)
B-5		(A ₁ ,1)	(A ₁ ,1)
C-4			
D-3			
E-2			(A ₂ ,2)

A despierta, pero ya hay líder, así que se vuelve seguidor.

Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**

 **Nodo caído**

Term	1	2	3
A-1 	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)
B-5 	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)
C-4 			
D-3 			
E-2 		(A ₂ ,2)	(A ₂ ,2)

E se cae temporalmente.

A recupera liderazgo por menor *timeout* y su último *log* es más actualizado que C y D.

Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**

Term	1	2	3
A-1 	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)
B-5 	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)
C-4 			(A ₁ ,1)
D-3 			(A ₁ ,1)
E-2 		(A ₂ ,2)	(A ₂ ,2)

A replica su *log* a C y D.

¿A₁ se considera consolidado?

Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**

<i>Term</i>	1	2	3
A-1 	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)
B-5 	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)
C-4 			(A ₁ ,1)
D-3 			(A ₁ ,1)
E-2 		(A ₂ ,2)	(A ₂ ,2)

¿A₁ se considera consolidado?

No, estamos en *term* 3, y esta acción fue creada en *term* 1.

Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**

Term	1	2	3
A-1 	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)
B-5 	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)
C-4 			(A ₁ ,1)
D-3 			(A ₁ ,1)
E-2 		(A ₂ ,2)	(A ₂ ,2)

¿A₁ se considera consolidado?

No, estamos en *term* 3, y esta acción fue creada en *term* 1.

Incluso... puede pasar lo siguiente

Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**

<i>Term</i>	1	2	3	4
A-1 	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)
B-5 	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)
C-4 			(A ₁ ,1)	(A ₁ ,1)
D-3 			(A ₁ ,1)	(A ₁ ,1)
E-2 		(A ₂ ,2)	(A ₂ ,2)	(A ₂ ,2), (A ₄ ,4)



Nodo caído

A se cae permanentemente.

E se vuelve líder porque tiene menor *timeout* y su último *log* tiene el *term* más actual.

Le llega una nueva acción (A₄)

Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**

Term	1	2	3	4
A-1 	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)
B-5 	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)
C-4 			(A ₁ ,1)	(A ₂ ,2), (A ₄ ,4)
D-3 			(A ₁ ,1)	(A ₂ ,2), (A ₄ ,4)
E-2 		(A ₂ ,2)	(A ₂ ,2)	(A ₂ ,2), (A ₄ ,4)



Nodo caído

E replica su *logs* a 2 nodos activos

Ahora A₁ ni siquiera está en la mayoría, pero A₄ sí.

A₄ se **consolida directamente**

¿Y qué pasa con A₂? 🤔

Raft - Fase 2 - Replicación de *logs*

Consolidación (directa)

- ◆ Una acción/operación es consolidada solo si esta fue replicada **a la mayoría de los nodos en el mismo mandato que fue creada.**

Consolidación (indirecta)

- ◆ Una acción/operación es consolidada indirectamente si existe una operación del mismo *log* que fue consolidada **directamente.**

Raft - Fase 2 - Replicación de *logs*

Consolidación (indirecta)

- Una acción/operación es consolidada indirectamente si existe una operación del mismo *log* que fue consolidada **directamente**.

 **Nodo caído**

Term	1	2	3	4
A-1 	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)
B-5 	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)
C-4 			(A ₁ ,1)	(A ₂ ,2), (A ₄ ,4)
D-3 			(A ₁ ,1)	(A ₂ ,2), (A ₄ ,4)
E-2 		(A ₂ ,2)	(A ₂ ,2)	(A ₂ ,2), (A ₄ ,4)

A₄ se **consolida directamente**

¿Y qué pasa con A₂? 🤔

Raft - Fase 2 - Replicación de *logs*

Consolidación (indirecta)

- Una acción/operación es consolidada indirectamente si existe una operación del mismo *log* que fue consolidada **directamente**.

Term	1	2	3	4
A-1 	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)
B-5 	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)	(A ₁ ,1)
C-4 			(A ₁ ,1)	(A ₂ ,2), (A ₄ ,4)
D-3 			(A ₁ ,1)	(A ₂ ,2), (A ₄ ,4)
E-2 		(A ₂ ,2)	(A ₂ ,2)	(A ₂ ,2), (A ₄ ,4)



Nodo caído

A₄ se **consolida directamente**

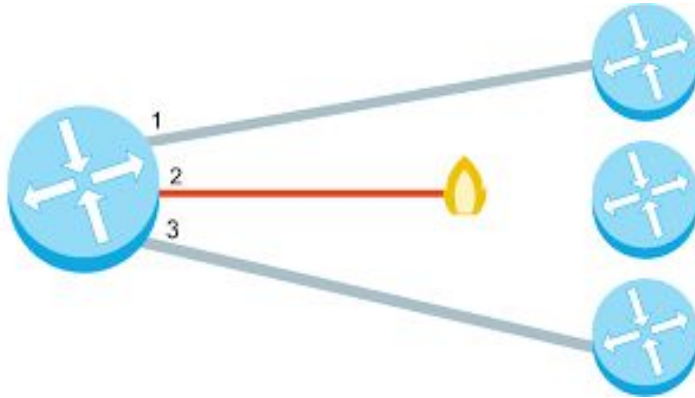
¿Y qué pasa con A₂? 🤔
Fue consolidada indirectamente.

Mientras exista mayoría de nodos (requisito del algoritmo), el líder será C, D o E que tienen A₂ y A₄.

Raft - Garantías (las mismas que Paxos)

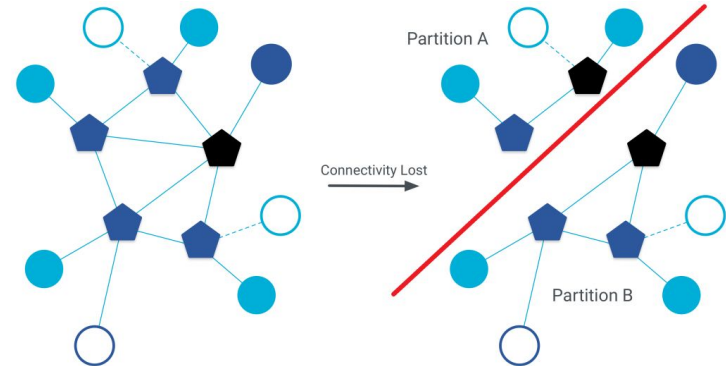
◆ Tolerancia a fallos

Pueden fallar $(N-1) // 2$ nodos.



◆ Tolerancia a particiones.

Si se divide la red en 2 grupos, mientras uno tenga la mayoría, el algoritmo funciona.



Raft - Vida real

- ◆ **etcd** (base de datos clave-valor) utiliza Raft para gestionar su log replicado; es la base de Kubernetes, Docker Swarm, entre otros.
- ◆ **Neo4j** (base de datos de grafos) garantiza consistencia y seguridad mediante Raft. eBay ocupa Neo4j.
- ◆ **ScyllaDB** (base de datos distribuida NoSQL) usa Raft internamente para metadata y transacciones. Discord y Disney+ ocupan ScyllaDB.

Poniendo a prueba lo que hemos aprendido

Respecto a la consolidación de datos utilizando el algoritmo Raft visto en clases, ¿cuál de las siguientes afirmaciones es **incorrecta**?

- I. Una operación que no fue consolidado durante un *term* nunca podrá consolidarse en mandatos posteriores.
 - II. Si una operación logra almacenarse en la mayoría de los nodos luego de diferentes mandatos, se garantiza que dicha operación será consolidado.
 - III. Si una operación se almacena en la mayoría de los nodos durante el mismo *term* este será consolidado.
 - IV. Es posible consolidar más de una operación en un mismo *term*.
- a. Solo I
 - b. Solo II
 - c. I y II
 - d. III y IV
 - e. I, II y III

Poniendo a prueba lo que hemos aprendido

Respecto a la consolidación de datos utilizando el algoritmo Raft visto en clases, ¿cuál de las siguientes afirmaciones es **incorrecta**?

- I. Una operación que no fue consolidado durante un *term* nunca podrá consolidarse en mandatos posteriores.
- II. Si una operación logra almacenarse en la mayoría de los nodos luego de diferentes mandatos, se garantiza que dicha operación será consolidado.
- III. Si una operación se almacena en la mayoría de los nodos durante el mismo *term* este será consolidado.
- IV. Es posible consolidar más de una operación en un mismo *term*.

- a. Solo I
- b. Solo II
- c. I y II**
- d. III y IV
- e. I, II y III

Próximos eventos

Próxima clase

- ◆ Nueva unidad: Fiabilidad en Sistemas Distribuidos
- ◆ Partiremos profundizando sobre la tolerancia a fallos. ¿Qué tipos de fallas existen? ¿Qué otras técnicas hay para sobrellevar o detectar fallos?

Evaluación

- ◆ **Control 9 - Obligatorio** - 2 preguntas sobre la clase de hoy. Se puede responder hasta este viernes a las 20:00.
- ◆ **AC05 Formativa** - Automatizar algoritmo de elección de líder bajo la lógica de Raft. Buzón abierto hasta el domingo 18 de octubre.

IIC2523

Sistemas Distribuidos

— Hernán F. Valdivieso López —
(2026 - 2 / Clase 09)

Créditos (animes utilizados)

Oshi no Ko



One Piece



Sword Art Online



Shingeki no Kyojin



Boku no Hero Academia



Konosuba

